

TDMM: An R Package for Bayesian Time-Dynamic Mixed-Effects Modeling of Longitudinal Data

Makena Grigsby¹ and Esra Kürüm¹

¹Department of Statistics, University of California, Riverside

Abstract

Longitudinal studies are widely used to examine changes in outcomes over time, but standard regression models typically assume that covariate effects remain constant throughout follow-up. This assumption may be restrictive when covariate-response relationships strengthen, weaken, or change direction over the observation period. Time-dynamic mixed-effects models (TDMM) address this setting by combining smoothly varying coefficient functions with subject-specific random effects that account for within-subject dependence. The TDMM package provides an R implementation of Bayesian time-dynamic mixed-effects models for regularly observed longitudinal data with baseline covariates. The current implementation supports Gaussian, Bernoulli, and Poisson outcomes and uses spline-based representations of time-varying coefficient functions with Markov chain Monte Carlo (MCMC) estimation through Just Another Gibbs Sampler (JAGS). This tutorial describes the statistical model, Bayesian estimation framework, package implementation, data requirements, fitting functions, MCMC diagnostics, visualization tools, and implementation examples for Gaussian and Bernoulli outcomes, including a real-data application.

Keywords: Bayesian modeling; longitudinal data; mixed-effects models; P-splines; time-varying coefficient functions.

1 Introduction

Analysis of longitudinal data is common in biomedical, behavioral, and social science research, where repeated measurements are used to study changes in outcomes over time. In many applications, however, interest is not limited to the response trajectory itself. Researchers may also seek to understand how covariate-response relationships evolve during follow-up. Standard regression models typically represent these relationships using time-constant regression coefficients, which may obscure changes in the magnitude or direction of covariate effects over the observation period.

Varying-coefficient models (Cai et al., 2000; Kürüm et al., 2014; Franco-Villoria et al., 2019) provide a flexible framework for studying such dynamic associations by allowing regression coefficients to vary smoothly as functions of an index variable. The index variable may represent time, age, spatial location, or another scientifically meaningful ordering variable (Franco-Villoria et al., 2019). In longitudinal studies, follow-up time provides a natural index, so the coefficient functions describe how covariate effects change over the course of observation. This formulation preserves the interpretability of regression modeling while allowing covariate associations to strengthen, weaken, or change direction over time. It is also closely related to time-varying effect modeling, where regression coefficients are estimated as functions of time (Shiyko et al., 2012; Lanza et al., 2016).

For longitudinal data, however, modeling time-varying covariate effects is not sufficient by itself, because repeated measurements from the same subject are correlated. Time-dynamic mixed-effects models (TDMM) address this by combining smoothly varying regression coefficient functions with subject-specific random effects that account for within-subject dependence.

The `TDMM` package provides an R implementation of Bayesian time-dynamic mixed-effects models for regularly observed longitudinal data with baseline covariates. The current version is designed for a structured longitudinal setting in which subjects are observed on a common time grid and covariates are measured at the subject level. The package currently supports Gaussian, Bernoulli, and Poisson outcomes. The package organizes the modeling workflow through functions for data checking, model fitting, posterior summarization, Markov chain Monte Carlo (MCMC) diagnostics, and visualization of estimated time-varying coefficient functions.

This tutorial is structured as follows. Section 2 introduces the time-dynamic mixed-effects models and Bayesian estimation framework. Section 3 describes the package implementation, including installation, data requirements, model fitting, parallel computation, diagnostics, and visualization. Section 4 presents implementation examples for Gaussian and Bernoulli outcomes, including a real-data application using the `WageData` dataset from the `panelr` R package (Long, 2020). Section 5 summarizes the main package functions. Section 6 discusses common issues that may arise during package use, and Section 7 outlines possible future extensions.

2 Time-dynamic mixed-effects models

2.1 Model specification

We begin by introducing notation. Let $Y_i(t)$ denote the longitudinal outcome for subject i , $i = 1, \dots, n$, at time t . In the current implementation, each subject is observed at time points $t = t_j$, $j = 1, \dots, m$, where subjects are assumed to share a common time grid, $t_1, \dots, t_m$.

A generalized time-dynamic mixed-effects model is defined as

$$g\{\mu_i(t)\} = \eta_i(t) = \mathbf{x}_i^\top \boldsymbol{\beta}(t) + b_i,$$

where $\mu_i(t) = E\{Y_i(t) \mid \mathbf{x}_i, b_i\}$, $g(\cdot)$ is the link function, and $\eta_i(t)$ is the linear predictor. Here, $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})^\top$ denotes the vector of baseline predictors, with $x_{i1} = 1$ corresponding to the intercept, and $\boldsymbol{\beta}(t) = \{\beta_1(t), \dots, \beta_p(t)\}^\top$ is the vector of time-varying coefficient functions. Thus, $\beta_1(t)$ represents the time-varying intercept function, while $\beta_k(t)$, for $k = 2, \dots, p$, represents the time-varying effect of the corresponding baseline predictor. The term b_i is a subject-specific random intercept that accounts for within-subject dependence among repeated observations from the same subject and is assumed to follow $b_i \sim N(0, \sigma_b^2)$ independently across subjects.

2.2 Estimation

We propose to estimate the model parameters using a Bayesian framework. Let $\boldsymbol{\theta}(t) = \{\boldsymbol{\beta}(t), \boldsymbol{\sigma}\}^\top$ denote the collection of unknown model components, where $\boldsymbol{\beta}(t) = \{\beta_1(t), \dots, \beta_p(t)\}^\top$ contains the time-varying coefficient functions. For Gaussian outcomes, $\boldsymbol{\sigma} = (\sigma_b^2, \sigma_\epsilon^2)^\top$, where σ_b^2 is the random-intercept variance and σ_ϵ^2 is the residual variance. For Bernoulli and Poisson outcomes, $\boldsymbol{\sigma} = \sigma_b^2$, since their conditional variability is determined by the corresponding response distribution and therefore no separate residual variance is included. Let $\mathbf{b} = (b_1, \dots, b_n)^\top$ denote the vector of subject-specific random intercepts. Given the observed longitudinal responses $\mathbf{Y}$, Bayesian inference is based on

$$p\{\boldsymbol{\theta}(t), \mathbf{b} \mid \mathbf{Y}\} \propto p\{\mathbf{Y} \mid \boldsymbol{\theta}(t), \mathbf{b}\} p\{\mathbf{b}, \boldsymbol{\theta}(t)\} = p\{\mathbf{Y} \mid \boldsymbol{\theta}(t), \mathbf{b}\} p\{\mathbf{b} \mid \boldsymbol{\theta}(t)\} p\{\boldsymbol{\theta}(t)\}.$$

Here, $p\{\mathbf{Y} \mid \boldsymbol{\theta}(t), \mathbf{b}\}$ is the likelihood induced by the selected response model, $p\{\mathbf{b} \mid \boldsymbol{\theta}(t)\}$ is the distribution of the subject-specific random intercepts, and $p\{\boldsymbol{\theta}(t)\}$ is the prior distribution for the remaining model components.

Under this estimation framework, the time-varying coefficient functions are represented using penalized splines (P-splines) (Eilers and Marx, 1996). Let $B_1(t), \dots, B_q(t)$ denote spline basis functions evaluated at time t . For $k = 1, \dots, p$, the coefficient function $\beta_k(t)$ is written as

$$\beta_k(t) = \sum_{\ell=1}^q B_\ell(t) \alpha_{k\ell},$$

where $\alpha_{k\ell}$ is the spline coefficient associated with the ℓ th basis function for the k th time-varying coefficient. Equivalently, with $\mathbf{B}(t) = \{B_1(t), \dots, B_q(t)\}^\top$ and $\boldsymbol{\alpha}_k = (\alpha_{k1}, \dots, \alpha_{kq})^\top$, this can be written as

$$\beta_k(t) = \mathbf{B}(t)^\top \boldsymbol{\alpha}_k.$$

This representation allows each coefficient function to vary flexibly over time while reducing estimation of the functional effects to estimation of a finite-dimensional set of spline coefficients.

To regularize the fitted curves, the spline coefficients are assigned smoothing priors based on differences between adjacent coefficients. In the current implementation, smoothness is encouraged through a second-order difference penalty, which discourages abrupt local changes in the fitted coefficient functions. Equivalently, the smoothing prior can be written in terms of a penalty matrix $\mathbf{K}$ as

$$\boldsymbol{\alpha}_k \mid \tau_k \sim N\{\mathbf{0}, (\tau_k \mathbf{K})^-\},$$

where $(\cdot)^-$ denotes a generalized inverse and τ_k is the smoothing precision parameter. Larger values of τ_k impose stronger smoothing, while smaller values allow the corresponding coefficient function to vary more freely. This P-spline formulation balances flexibility with protection against overfitting (Eilers and Marx, 1996; Lang and Brezger, 2004; Franco-Villoria et al., 2019). The smoothing precision parameters are assigned Gamma priors such that $\tau_k \sim \text{Gamma}(a_{\tau_k}, b_{\tau_k})$, $k = 1, \dots, p$. The subject-specific random intercepts are modeled as $b_i \mid \tau_b \sim N(0, \tau_b^{-1})$, $i = 1, \dots, n$, where $\tau_b = 1/\sigma_b^2$ is the random-intercept precision and is assigned a Gamma prior such that $\tau_b \sim \text{Gamma}(a_b, b_b)$. For Gaussian outcomes, the residual precision is similarly assigned $\tau_\epsilon \sim \text{Gamma}(a_\epsilon, b_\epsilon)$.

Posterior computation is performed using MCMC through Just Another Gibbs Sampler (JAGS) (Plummer, 2017). The resulting posterior samples are used to compute posterior means, posterior standard deviations, and pointwise credible intervals for the time-varying coefficient functions and other model parameters.

3 Package implementation and usage

The TDMM package is designed to provide a unified workflow for fitting time-dynamic mixed-effects models to longitudinal data in which all subjects are observed at the same set of time points. This package workflow is organized around four main tasks: preparing data, checking that the data satisfy the required structure, fitting the model, and inspecting the fitted output. The same general workflow is used for Gaussian, Bernoulli, and Poisson outcomes, with family specific likelihoods and response checks applied where needed.

The main fitting function is `tdmm()`. A parallel version, `tdmm.parallel()`, is also provided for running MCMC chains in parallel when multiple cores are available. Additional helper functions are used to check the data structure, examine family-specific response behavior, summarize fitted objects, and visualize posterior estimates of the time-varying coefficient functions.

3.1 Installation and dependencies

As the **TDMM** package relies on JAGS for Bayesian MCMC estimation, users should first install JAGS from the official website before fitting models (Plummer, 2017). The R interface to JAGS is handled through **rjags** and **R2jags**, while posterior samples are stored and summarized using **coda**, and selected diagnostic and plotting workflows may use **mcmcplots** packages (Plummer, 2025; Su and Yajima, 2024; Plummer et al., 2006; Curtis, 2018).

The required R packages can be installed using:

```
R> install.packages(c("rjags", "R2jags", "coda", "R2WinBUGS", "mcmcplots"))
```

The **TDMM** package can be installed from GitHub using:

```
R> devtools::install_github("esrakurum/Bayesian-TDMM")
```

The package also uses the **parallel** package, which is included with base R, for chain-level parallel computation in **tdmm.parallel()**.

3.2 Data requirements

The package expects the input data to be provided as a data frame in long format. Each row should represent one observation for one subject at one time point. At minimum, the data should include a subject identifier, a time variable, a response variable, and one or more baseline covariates.

As an example, the required structure can be described as:

subject.ID	time	y	x1	x2	...	xp
1	0.00	.	.	.	...	.
1	0.05	.	.	.	...	.
1	0.10	.	.	.	...	.
2	0.00	.	.	.	...	.
2	0.05	.	.	.	...	.
2	0.10	.	.	.	...	.

Here, **subject.ID** identifies the subject, **time** gives the repeated measurement time, **y** is the response variable, and **x1**, **x2**, ..., **xp** are baseline (time-invariant) covariates.

The current implementation assumes a balanced repeated-measures structure. Each subject has repeated measurements over time, all subjects share the same time grid, and baseline covariates are constant within each subject. The rows should be ordered by subject identifier and then by time. This ordering is important because the fitting functions use a subject-level indexing structure.

The main data-checking function is **check.tdmm.family.data()**. Users should run this function before model fitting, specifying the response family that will be used in **tdmm()**. This function calls **check.tdmm.data.structure()** to check the general data structure, including the required columns, subject ordering, repeated-measures structure, common time grid, and baseline-covariate consistency. It then calls the appropriate family-specific response check: **check.gaussian.response()**

for Gaussian outcomes, `check.bernoulli.balance()` for Bernoulli outcomes, and `check.poisson.counts()` for Poisson outcomes.

Section 4 illustrates the use of these functions in applications of the TDMM package to analyze Gaussian and Bernoulli outcomes. Once the data are prepared in the required structure, models can be fit using the main function `tdmm()` or the parallel version `tdmm.parallel()`.

3.3 Fitting a model with `tdmm()`

The function `tdmm()` is the main fitting function in the package. It provides a unified interface for fitting Bayesian time-dynamic mixed-effects models across the supported outcome families. After the data have been prepared in the required long format, the user specifies the dataset, response family, number of spline basis functions, and MCMC settings. The function then constructs the model inputs, fits the corresponding Bayesian model, and returns a fitted object containing posterior samples and model summaries.

A typical call to `tdmm()` is:

```
R> fit.gauss <- tdmm(data = data.gauss, family = "gaussian", nknots = 15,  
+ n.chains = 3, n.iter = 10000, n.burn = 3000, n.thin = 5, n.adapt = 5000)
```

The argument `data` is the prepared longitudinal data frame. The argument `family` specifies the response family and determines the likelihood and link function used in the model. The current options are “gaussian”, “bernoulli”, and “poisson”. The argument `nknots` specifies the number of knots used to construct the cubic spline basis for the time-varying coefficient functions. The remaining arguments control the MCMC procedure, including the number of chains (`n.chains`), total iterations (`n.iter`), burn-in iterations (`n.burn`), thinning interval (`n.thin`), and adaptation period (`n.adapt`). The values shown above reflect the default MCMC settings; if not specified by the user, `tdmm()` will use these values automatically.

The fitted object returned by `tdmm()` contains posterior samples, estimated time-varying coefficient functions, variance summaries, and model settings. The posterior samples, stored in `post.samples`, are used by the diagnostic and plotting functions described in Sections 3.5 and 3.6, respectively.

A simple first check after fitting the model is:

```
R> names(fit.gauss)
```

which allows the user to verify that the fitted object contains the main components returned by `tdmm()`. Table 1 summarizes the main output components.

Output component	Description
<code>post.samples</code>	Posterior samples returned by JAGS.
<code>beta.hat</code>	Matrix of posterior mean coefficient function estimates including the intercept function.
<code>beta.names</code>	Names of the estimated coefficient functions.
<code>time.points</code>	Time grid used for estimation in the model.
<code>model.settings</code>	Family, spline basis matrix, and MCMC settings.
<code>sigma2.b</code>	Estimated random-intercept variance.
<code>sigma2.e</code>	Estimated residual variance for Gaussian outcomes.

Table 1: Main components returned by `tdmm()`.

For Gaussian outcomes, the fitted object includes both random-intercept variance, `sigma2.b`, and residual variance, `sigma2.e`. For Bernoulli and Poisson outcomes, the fitted object includes only the random-intercept variance, `sigma2.b`. More detailed output interpretation is provided in the implementation examples.

3.4 Parallel fitting with `tdmm_parallel()`

The function `tdmm_parallel()` provides a parallel version of the main fitting workflow. It fits the same Bayesian time-dynamic mixed-effects model as `tdmm()`, but runs MCMC chains in parallel when multiple cores are available. This can reduce computation time, especially for longer MCMC runs, larger data sets, or simulation studies that require fitting the model many times.

A typical call to `tdmm_parallel()` is:

```
R> fit.gauss.par <- tdmm_parallel(data = data.gauss, family = "gaussian",
+ nknots = 15, n.chains = 3, n.iter = 10000, n.burn = 3000, n.thin = 5,
+ n.adapt = 5000, n.cores = 3)
```

The arguments are the same as those used in `tdmm()`, with the addition of `n.cores`, which specifies the number of cores used for parallel computation. The values shown above reflect the default MCMC settings and will be used automatically if not specified by the user. The current implementation uses `mclapply()` to distribute MCMC chains across cores. This approach works naturally on macOS, Linux, and Linux-based HPC systems, but may be limited on Windows. In most cases, `n.cores` should be no larger than `n.chains`, since the parallel workflow is intended to distribute MCMC chains across available cores. The object returned by `tdmm_parallel()` has the same general structure as the object returned by `tdmm()`. It contains posterior samples, estimated time-varying coefficient functions, variance summaries, processed inputs, and model settings. Therefore, the same diagnostic, summary, and plotting tools described in Section 3.3 can be applied directly to the object returned by `tdmm_parallel()`.

In practice, the parallel version is most useful after the model has already been checked on a smaller run. A recommended workflow is to first run `tdmm()` with shorter MCMC settings to

confirm that the data structure, response family, and model inputs are correct. Once the model runs successfully, `tdmm.parallel()` can be used for longer runs or repeated simulation settings.

3.5 MCMC diagnostics

After fitting a TDMM model, users should examine the MCMC output before interpreting posterior summaries. The estimated coefficient functions, variance components, and credible intervals are computed from posterior samples, so their reliability depends on adequate mixing of the chains.

The function `plot.trace.tdmm()` can be used to produce traceplots for selected posterior parameters from a fitted object. For example, the user may first define a vector of posterior parameters and then pass this vector to `plot.trace.tdmm()`:

```
R> params <- c("sigma2.e", "sigma2.b")
R> plot.trace.tdmm(result = fit.gauss, params = params)
```

Here, the `params` argument specifies the posterior parameters selected for the traceplots. Users can choose other parameters by checking the names in `colnames(fit.gauss$post.mat)`. Traceplots for variance components and selected spline coefficients can be especially useful, since these parameters determine the fitted time-varying coefficient functions.

Traceplots provide a visual diagnostic for assessing mixing. For a well-mixed parameter, the sampled values should fluctuate around a stable range without clear trends, long periods of sticking, or persistent separation among chains. If the traceplots show these problematic features, the posterior summaries may not be reliable. Guidance on possible next steps, including increasing `n.iter`, `n.burn`, or `n.adapt`, is provided in Section 6.2.

3.6 Visualization of estimated coefficient functions

After fitting the model and checking the MCMC output, the estimated time-varying coefficient functions can be visualized using `plot.tdmm()`, which uses the original data and fitted object to obtain posterior summaries of the coefficient functions over a plotting grid. A typical call is:

```
R> plot.tdmm(data = data.gauss, result = fit.gauss, sd = FALSE,
+ level = 0.95, grid = NULL, n.grid = 100)
```

The argument `data` is the longitudinal data frame used to fit the model, and `result` is the fitted object returned by `tdmm()` or `tdmm.parallel()`. The argument `sd` controls whether posterior standard deviation bands are displayed instead of pointwise credible intervals. When `sd = FALSE`, `plot.tdmm()` displays posterior mean curves with pointwise credible intervals. These intervals summarize uncertainty in each estimated coefficient function at each plotted time point. When `sd = TRUE`, the function plots posterior mean curves with bands based on one posterior standard deviation above and below the posterior mean at each plotted time point. The argument `level` gives the credible interval level, with default 0.95, and is used when `sd = FALSE`. The argument

`grid` allows the user to supply a custom time grid for plotting. If `grid = NULL`, the function creates a grid automatically from the observed time range using `n.grid` points. The values shown in the typical call above reflect the default plotting settings; if not specified by the user, `plot.tdmm()` will use these values automatically.

For a model with p baseline covariates, the plotting output includes the estimated coefficient functions $\{\hat{\beta}_1(t), \dots, \hat{\beta}_p(t)\}$. The function $\hat{\beta}_1(t)$ represents the estimated time-varying intercept, while each $\hat{\beta}_k(t)$, for $k = 2, \dots, p$, represents the estimated time-varying effect of the corresponding baseline covariate.

The fitted curves should be interpreted together with the MCMC diagnostics described in Section 3.5. Guidance on assessing roughness, smoothness, boundary behavior, and the choice of `nknots` is provided in Section 6.

4 Implementation examples

This section illustrates the TDMM workflow for Gaussian and Bernoulli outcomes. Each example presents data checks using `check.tdmm.family.data()`, model fitting with `tdmm()`, inspection of the fitted object, MCMC diagnostics using `plot.trace.tdmm()`, visualization of the estimated coefficient functions using `plot.tdmm()`, and interpretation of results.

The examples follow the same general workflow, but the interpretation of the fitted coefficient functions depends on the response model and link function. For Gaussian outcomes, the coefficient functions are interpreted on the response scale, and for Bernoulli outcomes, on the log-odds scale.

4.1 Gaussian real-data example: Wage Data

The Gaussian real-data implementation uses the `WageData` dataset from the `panelr` package (Long, 2020). The data contain repeated wage measurements for individuals across annual survey waves from 1976 through 1982. In this example, the response is `lwage`, the natural logarithm of wages; we refer to this response as log wage throughout this section. A subset of the analysis dataset is shown below, where the first rows correspond to repeated measurements from the same individual.

```
R> head(wage_analysis)
  id t   lwage fem   ed year
1  1 1 5.56068   0    9 1976
2  1 2 5.72031   0    9 1977
3  1 3 5.99645   0    9 1978
4  1 4 5.99645   0    9 1979
5  1 5 6.06146   0    9 1980
6  1 6 6.17379   0    9 1981
```

Here, `lwage` is the Gaussian response, `fem` is the baseline gender indicator, and `ed` is the number of years of education. The variable `fem` is coded as 1 for female subjects and 0 for male subjects. The model time variable is the survey wave `t`; calendar year, stored as `year`, is used only for interpretation and plotting.

Before fitting the Gaussian TDMM, we summarize the observed longitudinal response by gender. Figure 1 shows the average log wage by gender across the seven survey waves, providing a descriptive visualization of the observed wage trajectories. According to this figure, the average log wage increases over time for both groups, with male subjects having higher average log wages than female subjects across all seven survey waves.

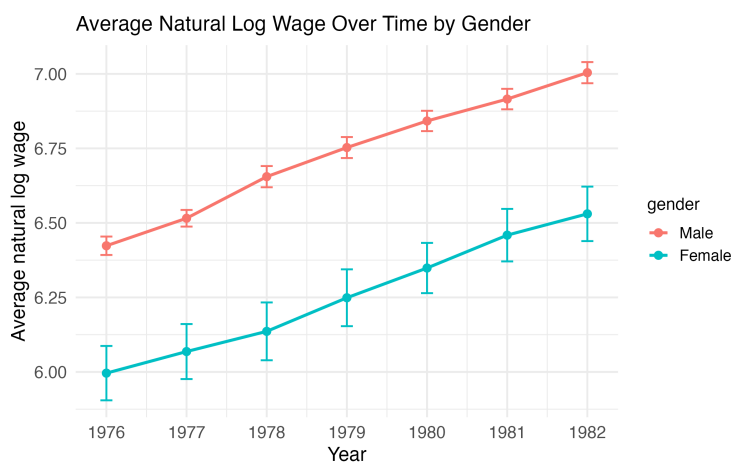


Figure 1: Average log wage over time by gender in the `WageData` dataset. Points show yearly means, and error bars show approximate 95% confidence intervals.

The analysis data are converted into the format expected by TDMM. The subject identifier is stored as `subject.ID`, the time variable as `time`, the response as `y`, and the baseline covariates as `x1` and `x2`. In this implementation, `x1` is the female indicator and `x2` is the number of years of education.

```
R> head(data.wage)
```

	subject.ID	time	y	x1	x2
1	1	1	5.56068	0	9
2	1	2	5.72031	0	9
3	1	3	5.99645	0	9
4	1	4	5.99645	0	9
5	1	5	6.06146	0	9
6	1	6	6.17379	0	9

The first rows of `data.wage` show one subject observed across multiple survey waves. The baseline covariates `x1` and `x2` remain constant across these repeated observations, consistent with the current TDMM requirement that baseline covariates are time-invariant within subject.

The prepared wage data are checked with `check.tdmm.family.data()` before model fitting. The `x.var` argument identifies the baseline covariates included in the model. For this example, this check verifies that the data have the repeated-measures structure, use a common time grid and have baseline covariates that remain constant within subject.

```
R> check.wage <- check.tdmm.family.data(data = data.wage, family = "gaussian",
+ x.var = c("x1", "x2"))
```

```
R> check.wage$structure.check$subject.balance$balanced
```

```
[1] TRUE
```

```
R> check.wage$structure.check$subject.balance$common.time.grid
```

```
[1] TRUE
```

```
R> check.wage$structure.check$x.constant.by.variable
```

```
x1    x2
```

```
TRUE TRUE
```

The checking output confirms that the prepared wage data satisfy the structural requirements of the current TDMM implementation. The data include subjects observed on a common time grid across the seven survey waves, and both baseline covariates, `x1` and `x2`, are constant within subject.

The Gaussian TDMM is then fit to the prepared wage data. The model includes three time-varying coefficient functions: an intercept function, a gender effect function, and an education effect function. Since the wage data contain seven observed survey waves, we use a relatively small number of knots with `nknots = 5`.

```
R> fit.wage <- tdmm(data = data.wage, family = "gaussian", nknots = 5,
+ x.var = c("x1", "x2"), n.chains = 3, n.iter = 10000, n.thin = 5,
+ n.adapt = 5000, n.burn = 3000)
```

```
Compiling model graph
```

```
  Resolving undeclared variables
```

```
  Allocating nodes
```

```
Graph information:
```

```
  Observed stochastic nodes: 4165
```

```
  Unobserved stochastic nodes: 603
```

```
  Total graph size: 35972
```

```
Initializing model
```

```
|*****| 100%
```

```
|*****| 100%
```

The JAGS output indicates that the model graph was compiled and initialized successfully. The fitted object is summarized using `summary.tdmm()`.

```
R> summary.tdmm(fit.wage)
```

```
TDMM fit summary
```

```
-----
```

```
Family: gaussian
```

```
Number of subjects: 595
```

```
Number of time points: 7
```

```
Total observations: 4165
```

```
Number of covariates: 2
```

```
Number of coefficient functions: 3
```

```
Covariates: x1, x2
```

```
MCMC settings
```

```
-----
```

```
Chains: 3
```

```
Iterations: 10000
```

```
Burn-in: 3000
```

```
Thinning: 5
```

```
Adaptation: 5000
```

```
Variance summaries
```

```
-----
```

```
sigma2.b: 0.09714004
```

```
sigma2.e: 0.02292527
```

```
Coefficient functions
```

```
-----
```

```
beta.hat
```

The fitted summary confirms that the model uses 595 subjects, 7 time points, and 4165 total observations. The two baseline covariates, `x1` and `x2`, produce three estimated coefficient functions: an intercept function and two covariate-effect functions. The covariate-effect functions are ordered according to `x.var`; in this example, the first covariate-effect function corresponds to the female indicator, and the second corresponds to years of education. For the Gaussian model, the summary also reports the random-intercept variance `sigma2.b` and residual variance `sigma2.e`.

Before interpreting the estimated coefficient functions, we inspect selected MCMC traceplots using `plot.trace.tdmm()`. These plots are used to assess whether the sampled values fluctuate

around a stable range without clear trends, long periods of sticking, or persistent separation among chains. For illustration, we examine the random-intercept variance `sigma2.b` and residual variance `sigma2.e`. These plots show no obvious trends or persistent separation among the chains, supporting the use of posterior summaries for these parameters.

```
R> params <- c("sigma2.b", "sigma2.e")
R> plot.trace.tdmm(result = fit.wage, params = params)
```

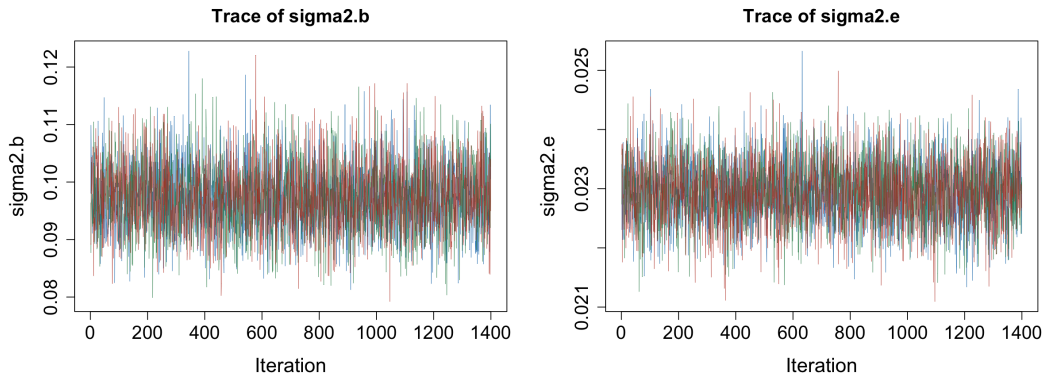


Figure 2: Traceplots for selected variance parameters from the Gaussian TDMM fit to `WageData`.

The estimated coefficient functions are visualized using `plot.tdmm()`. The model is fit using the survey wave variable `time`; calendar-year labels are added to the x-axis to facilitate interpretation in the context of the original `WageData` survey years.

```
R> plot.tdmm(data = data.wage, result = fit.wage, sd = FALSE, level = 0.95)
```

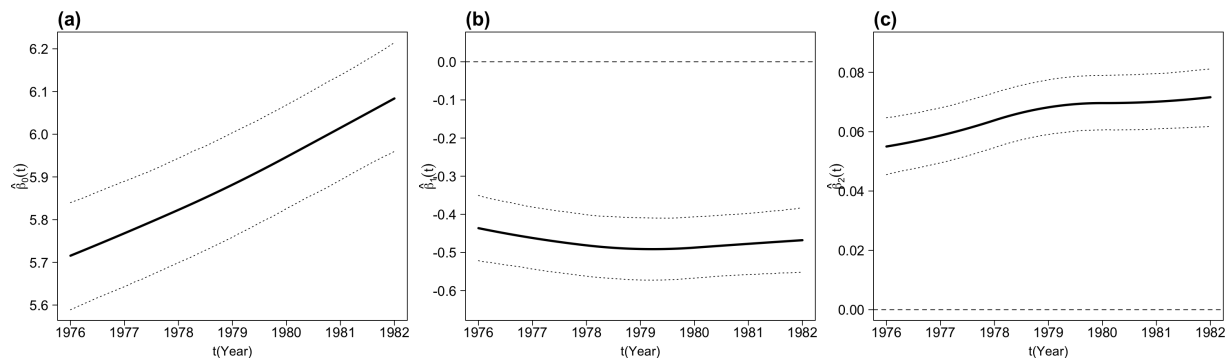


Figure 3: Estimated time-varying coefficient functions for the Gaussian TDMM fit to `WageData`: (a) intercept function, (b) female indicator effect, and (c) education effect. The solid curves show posterior mean estimates, and the dotted curves show pointwise 95% credible intervals.

Figure 3 displays the estimated time-varying coefficient functions from the Gaussian TDMM fit. Panel (a) shows the intercept function, Panel (b) shows the time-varying effect of the female indicator, and Panel (c) shows the time-varying effect of education.

According to Figure 3(a), the estimated intercept function increases steadily from 1976 to 1982, indicating an increasing average log wage trajectory over the survey period for the reference group, corresponding to male subjects with education. In Figure 3(b), the estimated coefficient function for `x1`, the female indicator, is negative across all survey years, and its pointwise credible intervals remain below zero. This suggests that, holding education fixed, female subjects have lower average log wages than male subjects throughout the study period. The magnitude of this negative association is fairly stable over time, with a slightly stronger negative effect around the middle survey years. Figure 3(c) shows the estimated coefficient function for `x2`, number of years of education, is positive across the survey period, with pointwise credible intervals above zero. This suggests that, holding gender fixed, additional years of education are associated with higher average log wages. The education effect appears to increase from 1976 through the late 1970s and then remains relatively stable, with only modest changes during the later survey years.

4.2 Bernoulli outcome

This Bernoulli example illustrates the TDMM workflow for a simulated binary longitudinal response. The data are generated using `data.gen.tdmm.bern()`, which creates repeated binary outcomes with a subject-specific random intercept and known time-varying coefficient functions. A subset of the simulated data is shown below, where the first rows show repeated observations for the same subject across time.

```
R> data.bern <- data.gen.tdmm.bern(n.subject = 200, n.time = 20,
+   sigma2.b = 0.5, seed = 541)
```

```
R> head(data.bern)
```

subject.ID	time	y	x1
1	1	0.00000000	1 0.9729502
2	1	0.05263158	1 0.9729502
3	1	0.10526316	1 0.9729502
4	1	0.15789474	0 0.9729502
5	1	0.21052632	0 0.9729502
6	1	0.26315789	1 0.9729502

```
R> table(data.bern$y)
```

0	1
2001	1999

The generated data follow the required long format, with one row for each subject at each time

point. The response y is binary, and the baseline covariate x_1 is repeated across time within each subject. The response table verifies that only 0 and 1 values are present and shows that the two outcome classes are approximately balanced overall.

The prepared data are checked with `check.tdmm.family.data()` before model fitting.

```
R> check.bern <- check.tdmm.family.data(data = data.bern, family = "bernoulli")
R> check.bern$structure.check$subject.balance$balanced
[1] TRUE
R> check.bern$structure.check$subject.balance$common.time.grid
[1] TRUE
R> check.bern$structure.check$x.constant.within.subject
[1] TRUE
R> check.bern$response.check$overall.proportions
y
      0      1
0.50025 0.49975
```

These checks confirm that the simulated Bernoulli data satisfy the required TDMM structure: repeated measurements are balanced, all subjects share a common time grid, and the baseline covariate is constant within subject. The response check confirms valid Bernoulli coding and shows that the two outcome classes are approximately balanced overall.

The Bernoulli TDMM is then fit using `tdmm()` with `family = "bernoulli"`. The likelihood and logit link are selected automatically from the response family. The model uses `nknots = 15`, and the remaining MCMC settings use the package defaults.

```
R> fit.bern <- tdmm(data = data.bern, family = "bernoulli", nknots = 15)
```

Compiling model graph

Resolving undeclared variables

Allocating nodes

Graph information:

Observed stochastic nodes: 4000

Unobserved stochastic nodes: 205

Total graph size: 37367

Initializing model

```
|*****| 100%
|*****| 100%
```

The JAGS output shows that the model was compiled and initialized successfully. The fitted object is summarized using `summary.tdmm()`.

```
R> summary.tdmm(fit.bern)

TDMM fit summary
-----
Family: bernoulli
Number of subjects: 200
Number of time points: 20
Total observations: 4000
Number of covariates: 1
Number of coefficient functions: 2
Covariates: x1

MCMC settings
-----
Chains: 3
Iterations: 10000
Burn-in: 3000
Thinning: 5
Adaptation: 10000

Variance summaries
-----
sigma2.b: 0.4749985
sigma.b: 0.68606

Coefficient functions
-----
beta.hat
```

The summary reports 200 subjects, 20 time points, and 4000 total observations. With one baseline covariate, `x1`, the model produces two estimated coefficient functions: an intercept function and one covariate-effect function.

Before interpreting the fitted Bernoulli coefficient functions, we inspect the MCMC traceplot for the random-intercept variance term. Figure 4 shows the traceplot for `sigma2.b`.

```
R> params <- "sigma2.b"
R> plot.trace.tdmm(result = fit.bern, params = params)
```

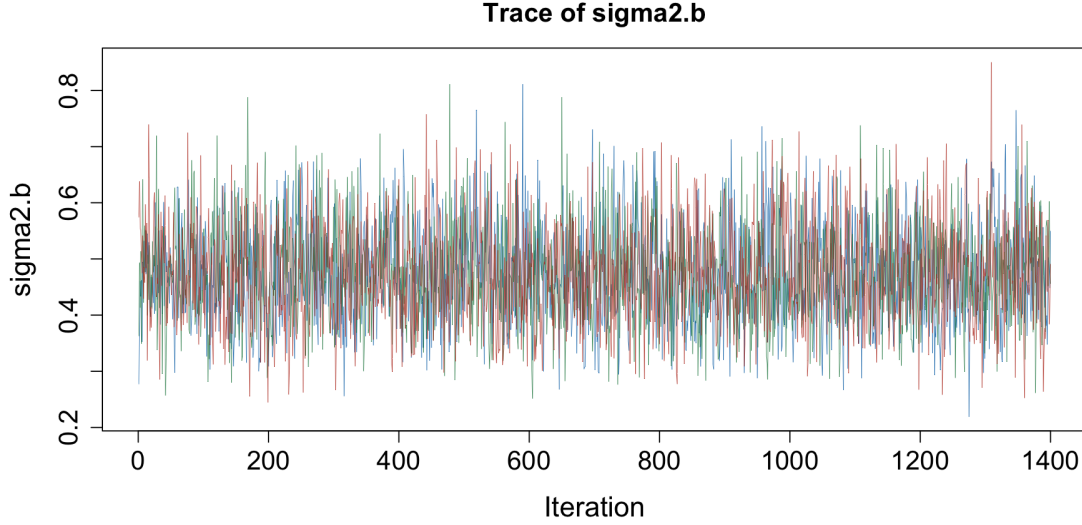


Figure 4: Traceplot for the random-intercept variance parameter from the Bernoulli TDMM.

The traceplot shows no obvious trends or persistent separation among the chains for `sigma2.b`, supporting the use of posterior summaries for this parameter.

The fitted coefficient functions are then visualized using `plot.tdmm()`, which displays posterior mean curves with pointwise credible intervals.

```
R> plot.tdmm(data = data.bern, result = fit.bern, sd = FALSE,
+ level = 0.95, grid = NULL, n.grid = 100)
```

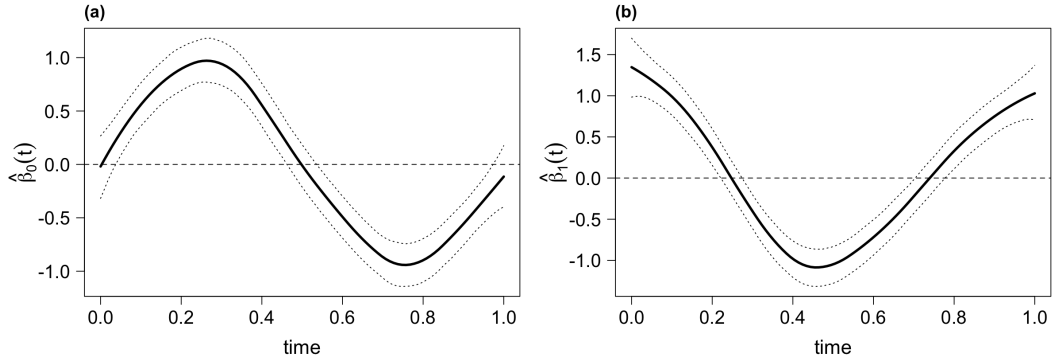


Figure 5: Estimated time-varying coefficient functions for the Bernoulli TDMM: (a) estimated intercept function and (b) estimated time-varying effect of `x1`. The solid curves show posterior mean estimates, and the dotted curves show pointwise 95% credible intervals.

Figure 5 shows the estimated time-varying coefficient functions from the Bernoulli TDMM fit. The left panel shows $\hat{\beta}_0(t)$, the estimated intercept function, and the right panel shows $\hat{\beta}_1(t)$, the estimated time-varying effect of `x1`. The dotted curves give the pointwise 95% credible intervals.

5 Summary of package functions

This section gives a compact reference for the main functions currently included in the package as well as the internal helper functions that support the fitting workflow. The goal is not to replace the fuller explanations in earlier sections, but to provide a quick lookup table that is easy to scan. The functions listed here focus on the main workflow for fitting TDMM to observed longitudinal data: checking the data, fitting the model, inspecting MCMC output, and visualizing the estimated coefficient functions.

5.1 Main fitting functions

Function	Purpose
<code>tdmm()</code>	Fits a time-dynamic mixed-effects model using the standard serial workflow.
<code>tdmm.parallel()</code>	Fits the same model using chain-level parallelization when multiple cores are available.

5.2 Data checking functions

The data checking functions are used before model fitting to verify that the input data satisfy the structure required by the current TDMM implementation. In most cases, users should call `check.tdmm.family.data()`, which runs the general structural checks and then applies the appropriate family-specific response check. The additional checking functions are useful for inspecting specific parts of the data structure, such as subject balance, common time grids, and baseline-covariate consistency.

Function	Purpose
<code>check.tdmm.family.data()</code>	Main data-checking function. Runs the general structural check and then calls the appropriate family-specific response check.
<code>check.tdmm.data.structure()</code>	Checks that the input data contain the required columns, are ordered by subject and time, use a common time grid, and have baseline covariates that are constant within subject.
<code>check.tdmm.subject.balance()</code>	Summarizes the repeated-measures structure, including the number of subjects, observations per subject, balance, and shared time grid.

5.3 Data generation functions

The package also includes data generation functions for simulating example data from the supported response families. These functions are useful for testing the package implementation, demonstrating the fitting workflow, and running simulation studies. Each function returns a longitudinal dataset

in the structure expected by TDMM, so the generated data can be used directly with the checking, fitting, diagnostic, and plotting functions.

Function	Purpose
<code>data.gen.tdmm.gauss()</code>	Generates simulated Gaussian longitudinal data from a TDMM structure.
<code>data.gen.tdmm.bern()</code>	Generates simulated Bernoulli longitudinal data from a TDMM structure.
<code>data.gen.tdmm.pois()</code>	Generates simulated Poisson longitudinal data from a TDMM structure.

5.4 Diagnostic and plotting functions

The diagnostic and plotting functions are used after fitting a model. The current plotting workflow is centered around checking MCMC output and visualizing the estimated time-varying coefficient functions for an observed data set.

Function	Purpose
<code>plot.trace.tdmm()</code>	Produces traceplots for selected posterior parameters from a fitted TDMM object.
<code>plot.tdmm()</code>	Plots posterior mean coefficient functions with pointwise credible intervals or posterior standard deviation bands.

5.5 Internal helper functions

Some package functions are mainly internal helpers used to support the fitting workflow. These functions are useful for understanding the package structure, but most users will interact mainly with the fitting, data checking, diagnostic, and plotting functions listed above.

Function	Purpose
<code>tpower()</code>	Defines the truncated power basis used in spline construction.
<code>bbase()</code>	Builds the spline basis matrix evaluated on a chosen time grid.
<code>get.tdmm.model.file()</code>	Locates the family-specific JAGS model file for the selected outcome family.
<code>build.tdmm.subject.inputs()</code>	Builds the subject-level input objects used by the fitting functions.
<code>build.tdmm.jags.data()</code>	Builds the JAGS data list from the processed subject-level inputs.
<code>get.tdmm.family.config()</code>	Selects the family-specific model file and monitored parameters.
<code>check.gaussian.response()</code>	Summarizes the Gaussian response overall and by time point.
<code>check.bernoulli.balance()</code>	Checks that the response is coded as 0/1 and summarizes response counts and proportions overall and by time point.
<code>check.poisson.counts()</code>	Checks that the response contains nonnegative integer counts and summarizes count behavior overall and by time point.
<code>recover.tdmm.betas()</code>	Recovers posterior mean estimates of the fitted coefficient functions.
<code>recover.tdmm.betas.ci()</code>	Recovers posterior summaries of the fitted coefficient functions, including posterior means, pointwise credible intervals, and posterior standard deviations.
<code>make.tdmm.plot.grid()</code>	Builds the time grid and spline basis used to evaluate fitted coefficient functions for plotting.
<code>build.tdmm.output()</code>	Constructs the unified output object returned by the fitting functions.

6 Troubleshooting and common issues

This section summarizes common issues that may arise when using the package. In many cases, fitting problems are related to the input data structure, the response behavior for the selected outcome family, the availability of the JAGS model files, or the MCMC settings. The goal of this section is to help users identify the source of an issue before interpreting the fitted coefficient functions.

6.1 Data structure and response checks

The data-checking functions described in Section 3.2 are intended to be used before fitting a TDMM model. If a model-fitting error occurs after these checks, users should return to the checking output

and identify which part of the data structure or response behavior failed.

Once the response family is specified, the combined structural and response check should be run using:

```
R> check.tdmm.family.data(data, family = "gaussian")
```

This function checks the repeated-measures structure, including subject ordering, balance, common time grid, and within-subject constancy of baseline covariates. It also applies response checks for the selected family. For example, Bernoulli responses should be coded as 0/1, and Poisson responses should be nonnegative integer counts.

The current implementation does not support time-varying covariates or unbalanced time grids. If either feature is present in the data, the dataset falls outside the current scope of the package and would require an extension of the model and fitting functions.

6.2 MCMC settings

The implementation examples in Section 4 use MCMC settings chosen to demonstrate the package workflow. For final analyses, users should first confirm that the data structure and response checks are satisfied, then fit a shorter test run, inspect the MCMC diagnostics, and increase the MCMC settings as needed. Longer runs may be needed when the dataset is larger, the spline basis is more flexible, or traceplots show clear trends, long periods of sticking, or persistent separation between chains. In these cases, users may consider increasing `n.iter` or `n.burn`. Runtime can also increase when `nknots` is large, because the model contains more spline coefficients.

The argument `n.adapt` controls the number of adaptation iterations used by JAGS before posterior sampling begins. During adaptation, JAGS tunes the samplers used for the model. If JAGS returns an “adaptation incomplete” warning, the model may still run, but the sampler may not have fully adapted. A common first step is to increase `n.adapt` and rerun the model.

6.3 Estimated coefficient functions

Estimated coefficient functions should be interpreted after checking the MCMC diagnostics and the data checks described earlier. Features such as wide credible intervals, rough fitted curves, nearly flat fitted curves, or instability near the boundaries of the time range should be examined before drawing substantive conclusions.

If the estimated coefficient functions appear unusually rough, users should first check the MCMC diagnostics for the spline coefficients and variance components. Poor mixing can make fitted curves appear unstable even when the model structure is reasonable. If the diagnostics are acceptable, excessive roughness may indicate that the spline basis is too flexible for the amount of information in the data. In that case, users may consider reducing `nknots` or comparing the fitted curves across a small number of reasonable `nknots` values.

If the estimated coefficient functions appear overly smooth or nearly flat, the selected `nknots` value may be too small to capture the time-varying pattern. Increasing `nknots` can provide addi-

tional flexibility, but larger spline bases increase the number of parameters and may require longer MCMC runs. When possible, the choice of `nknots` should be guided by sensitivity analyses or prediction-based criteria, such as cross-validation, rather than by selecting the curve shape that appears visually preferable.

Boundary behavior is common in spline-based estimation. Curves and uncertainty bands may be less stable near the beginning or end of follow-up because there is less neighboring information at the edges of the time domain. Users should avoid over-interpreting sharp changes near the boundaries unless the pattern is stable across reasonable `nknots` choices and supported by the observed data.

For Bernoulli outcomes, estimation may be more difficult when the response is highly imbalanced overall or at certain time points. If nearly all responses are 0 or 1 in part of the time domain, the fitted log-odds curves may have wider uncertainty bands or unstable behavior. For Poisson outcomes, sparse counts at certain time points or count variability much larger than the mean may also make the fitted model harder to interpret. In these cases, users should review the family-specific response checks before relying on the fitted curves.

7 Future extensions

The current version of the package provides a foundation for fitting Bayesian time-dynamic mixed-effects models for longitudinal data with baseline covariates. The current implementation allows the shared observation grid to be irregularly spaced in time, but assumes that all subjects are observed on the same grid and have the same number of repeated measurements. Future developments will aim to extend the package to more general longitudinal settings, including subjects with different numbers of observations or subject-specific observation times. Other potential extensions include additional response families and more flexible random-effects structures. Additional tools for MCMC diagnostics, posterior predictive checks, and visualization of fitted coefficient functions may also be added in future versions.

Software availability

The package repository is available on GitHub at: <https://github.com/esrakurum/Bayesian-TDMM>.

Disclosure

The authors acknowledges the use of AI-assisted tools to support editing. All final content was reviewed by the authors.

References

- Cai, Z., Fan, J., and Li, R. (2000). Efficient estimation and inferences for varying-coefficient models. *Journal of the American Statistical Association*, 95(451):888–902.
- Curtis, S. M. (2018). *mcmcplots: Create Plots from MCMC Output*. R package version 0.4.3.
- Eilers, P. H. C. and Marx, B. D. (1996). Flexible smoothing with B-splines and penalties. *Statistical Science*, 11(2):89–121.
- Franco-Villoria, M., Ventrucci, M., and Rue, H. (2019). A unified view on bayesian varying coefficient models. *Electronic Journal of Statistics*, 13:5334–5359.
- Kürüm, E., Li, R., Wang, Y., and Şentürk, D. (2014). Nonlinear varying-coefficient models with applications to a photosynthesis study. *Journal of Agricultural, Biological, and Environmental Statistics*, 19(1):57–81.
- Lang, S. and Brezger, A. (2004). Bayesian P-splines. *Journal of Computational and Graphical Statistics*, 13(1):183–212.
- Lanza, S. T., Vasilenko, S. A., and Russell, M. A. (2016). Time-varying effect modeling to address new questions in behavioral research: Examples in marijuana use. *Psychology of Addictive Behaviors*, 30(8):939–954.
- Long, J. A. (2020). *panelr: Regression Models and Utilities for Repeated Measures and Panel Data*. R package version 0.7.3.
- Plummer, M. (2017). *JAGS Version 4.3.0 User Manual*.
- Plummer, M. (2025). *rjags: Bayesian Graphical Models using MCMC*. R package version 4-17.
- Plummer, M., Best, N., Cowles, K., and Vines, K. (2006). Coda: Convergence diagnosis and output analysis for mcmc. *R News*, 6(1):7–11.
- Shiyko, M. P., Lanza, S. T., Tan, X., Li, R., and Shiffman, S. (2012). Using the time-varying effect model (tvem) to examine dynamic associations between negative affect and self confidence on smoking urges: Differences between successful quitters and relapsers. *Prevention Science*, 13:288–299.
- Su, Y.-S. and Yajima, M. (2024). *R2jags: Using R to Run JAGS*. R package version 0.8-9.